

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное бюджетное образовательное учреждение высшего образования
«Российский экономический университет имени Г.В. Плеханова»
Московский приборостроительный техникум

ОТЧЕТ

по учебной практике

УП 01.02 «Учебная практика»

индекс по УП и наименование практики

Профессионального модуля ПМ.01 «Разработка модулей программного обеспечения для компьютерных систем»

индекс по УП и наименование профессионального модуля

Специальность 09.02.07 «Информационные системы и программирование»

код и наименование специальности

Студент Афанасьев Александр Сергеевич

(фамилия, имя, отчество)

Группа П-2-24

Руководитель по практической подготовке от техникума

Юшина Дарья Денисовна

(фамилия, имя, отчество)

« » 2026 года

СОДЕРЖАНИЕ

Лабораторная работа №1	3
Лабораторная работа №2	5

Лабораторная работа №1

Цель работы: создать мини-приложение с карточкой товара и счётчиком.

1. ProductCard.jsx

Был создан компонент ProductCard в котором реализованы: разметка карточек товаров, счетчик, подтягивание информации через props.

```
App.jsx • ProductCard.jsx • App.css X
my-app > src > ProductCard.jsx > default
1 import {useState} from "react";
2
3 function ProductCard(props){
4   const [count,setCount]=useState(0);
5
6   return(
7     <div className="cardd">
8       <img src={props.image} alt={props.name}></img>
9       <h3>{props.name}</h3>
10      <p>Цена: {props.price} руб.</p>
11      {count===0 ? (<button onClick={()=>setCount(count+1)}>Добавить</button>):
12      (<div className="plusminus"><button onClick={()=>setCount(count+1)}>+</button> <p>{count} шт.</p> <button onClick={()=>setCount(count-1)}>-</button></div>)}
13    </div>
14  )
15 }
16 export default ProductCard;
```

Рисунок 1 – Создание карточек товаров

2. App.jsx

В App.jsx был реализован импорт необходимых компонентов (карточек товаров, компонента ProductCard),а так же вызов ProductCard с заполнением данных для карточек (название, цена, путь к картинке) о товарах.

```
Explorer
  my-app
    node_modules
    public
    src
      assets
        headphones.png
        hero.png
        notebook.png
        phone.png
        react.svg
        vite.svg
      App.css
      App.jsx
      Counter.jsx
      index.css
      main.jsx
      ProductCard.jsx
    .gitignore
    .oxlinttrc.json
    index.html
    package-lock.json
    package.json
    README.md
    vite.config.js

App.jsx • ProductCard.jsx X App.css X
my-app > src > App.jsx > ...
3 import './App.css'
4 import ProductCard from './ProductCard';
5 import notebook_image from './assets/notebook.png'
6 import headphones_image from './assets/headphones.png'
7 import phone_image from './assets/phone.png'
8
9 function App(){
10   return(
11     <div className="cardss">
12       <ProductCard
13         name="Phone"
14         price="89,999"
15         image={phone_image}
16       />
17
18       <ProductCard
19         name="Ноутбук"
20         price={100000}
21         image={notebook_image}
22       />
23
24       <ProductCard
25         name="Наушники"
26         price={5000}
27         image={headphones_image}
28       />
29     </div>
30   )
31 }
32
33
34
35
```

Рисунок 2 – Вызов компонента и заполнение карточек

3. App.css

Финальным этапом стало добавление минимальных стилей, а именно настройка отступов, размеров, цветов и т.п.

```
12  img{
13    height: 230px;
14    width: 300px;
15    border-radius:30px 30px 0 0;
16  }
17  .cardss {
18    display:flex;
19    margin: 50px;
20    padding:50px;
21    gap:50px;
22    justify-content: center;
23  }
24
25  .cardd {
26    height: 400px;
27    width: 300px;
28    background: #646161;
29    border-radius: 30px;
30    border:3px solid #a19898;
31  }
32
33  .plusminus{
34    display: flex;
35    gap:10px;
36    justify-content: center;
37  }
38  }
39
```

Рисунок 3 – Стили

4. Итоговый результат

На фото представлен итог проделанной работы.



Рисунок 4 - Итог

Вывод: было создано мини-приложение с карточкой товара и счётчиком.

Лабораторная работа №2

Цель работы: создать SPA-приложение на React с каталогом товаров, глобальной корзиной и сменой темы.

1. NatusVinsere.jsx

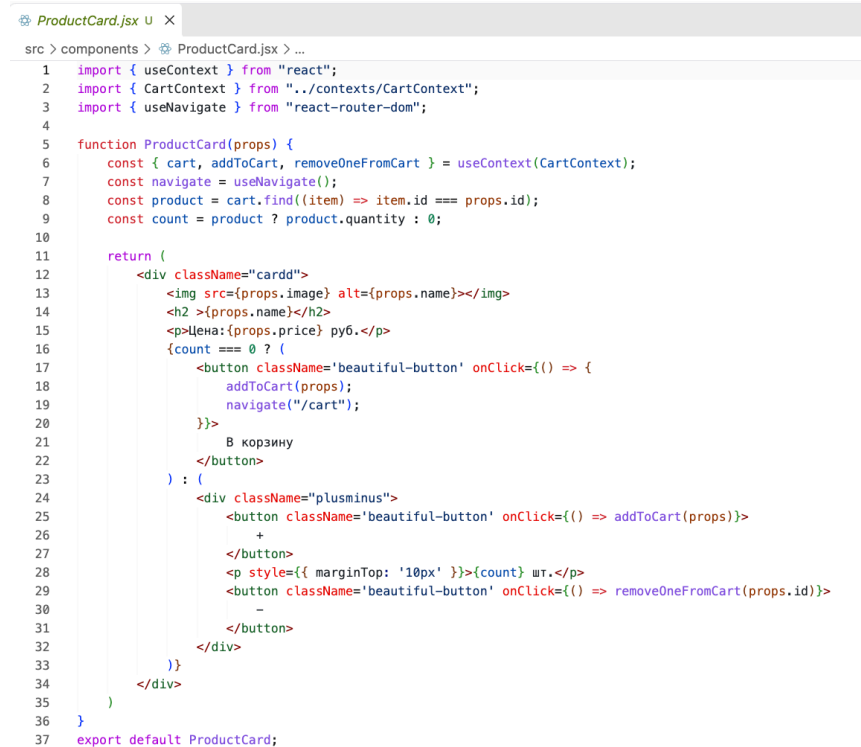
Был создан компонент NatusVinsere в котором реализовано меню навигации (NavLink) с изменяющимся состоянием в зависимости от активной страницы.

```
src > components > NatusVinsere.jsx > ...
1 import { NavLink } from 'react-router-dom';
2 import { useContext } from 'react';
3 import { ThemeContext } from '../contexts/ThemeContext';
4 import { CartContext } from '../contexts/CartContext';
5 import Switch from './Switcher';
6
7 function Navi() {
8   const { theme, toggleTheme } = useContext(ThemeContext);
9   const { cartCount } = useContext(CartContext);
10
11   return (
12     <div className="nav-bar">
13       <nav>
14         <NavLink
15           to="/"
16           style={({ isActive }) => ({
17             color: isActive ? "#9500ff" : "white",
18             fontWeight: isActive ? "bold" : "normal"
19           })
20         >
21           Главная
22         </NavLink>
23         <NavLink
24           to="/cart"
25           style={({ isActive }) => ({
26             color: isActive ? "#9500ff" : "white",
27             fontWeight: isActive ? "bold" : "normal"
28           })
29         >
30           Корзина <span className="cart-count">{cartCount}</span>
31         </NavLink>
32         <NavLink
33           to="/login"
34           style={({ isActive }) => ({
35             color: isActive ? "#9500ff" : "white",
36             fontWeight: isActive ? "bold" : "normal"
37           })
38         >
39           Вход
40         </NavLink>
41         <Switch theme={theme} toggleTheme={toggleTheme} />
42       </nav>
43     </div>
44   )
45 }
46 export default Navi
```

Рисунок 5 — Создание меню навигации

2. ProductCard.jsx

В компоненте ProductCard был немного изменен макет карточки с товаром, теперь изменение наличия товара в корзине происходит через глобальное состояние в контексте корзины.

The image shows a code editor window with the file 'ProductCard.jsx' open. The code defines a functional component 'ProductCard' that uses 'useContext' to interact with 'CartContext' and 'useNavigate' from 'react-router-dom'. It renders a card with an image, name, price, and a button to add to the cart. The button's click handler calls 'addToCart' and 'navigate' to the cart page. Below the button, it shows the current count of the item in the cart and buttons to increase or decrease the count using 'addToCart' and 'removeOneFromCart' respectively.

```
1 import { useContext } from "react";
2 import { CartContext } from "../contexts/CartContext";
3 import { useNavigate } from "react-router-dom";
4
5 function ProductCard(props) {
6   const { cart, addToCart, removeOneFromCart } = useContext(CartContext);
7   const navigate = useNavigate();
8   const product = cart.find((item) => item.id === props.id);
9   const count = product ? product.quantity : 0;
10
11   return (
12     <div className="cardd">
13       <img src={props.image} alt={props.name}></img>
14       <h2 >{props.name}</h2>
15       <p>Цена: {props.price} руб.</p>
16       {count === 0 ? (
17         <button className='beautiful-button' onClick={() => {
18           addToCart(props);
19           navigate("/cart");
20         }}>
21           В корзину
22         </button>
23       ) : (
24         <div className="plusminus">
25           <button className='beautiful-button' onClick={() => addToCart(props)}>
26             +
27           </button>
28           <p style={{ marginTop: '10px' }}>{count} шт.</p>
29           <button className='beautiful-button' onClick={() => removeOneFromCart(props.id)}>
30             -
31           </button>
32         </div>
33       )}
34     </div>
35   )
36 }
37 export default ProductCard;
```

Рисунок 6 - Макет карточки с товаром

3. CartContext.jsx и ThemeContext.jsx

Так же были созданы контексты для корзины с товарами и темы всего сайта.

The image shows two code editor windows side-by-side. The left window shows 'CartContext.jsx' with code to create and export 'CartContext'. The right window shows 'ThemeContext.jsx' with code to create and export 'ThemeContext' with a default value of 'light'.

```
CartContext.jsx
1 import { createContext } from "react";
2
3 export const CartContext = createContext();
4
ThemeContext.jsx
1 import { createContext } from 'react';
2
3 export const ThemeContext = createContext("light");
4
```

Рисунок 7 - контексты корзины и темы

4. CartProvider.jsx

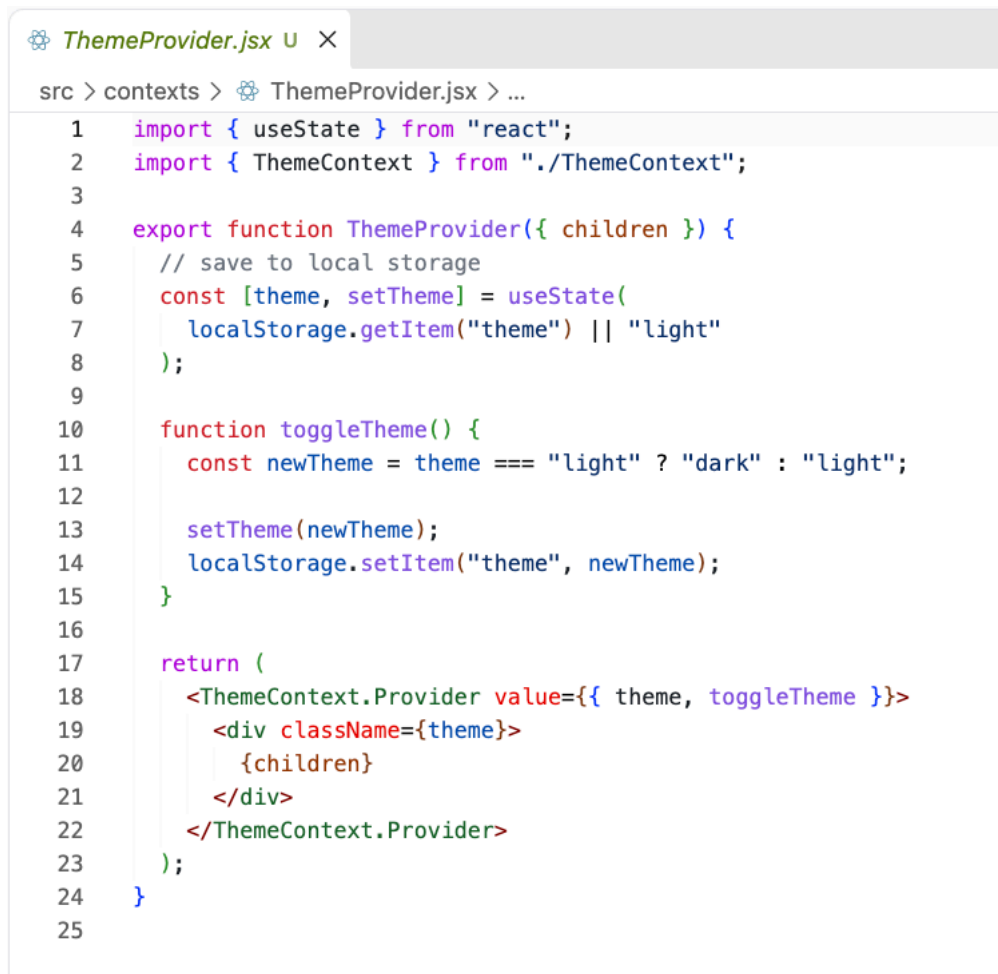
В CartProvider находится вся логика работы с товарами в корзине: добавление товара в корзину (с проверкой наличия товара в корзине и если товар есть то увеличение количества товара в корзине), удаление 1 ед. товара из корзины (уменьшение счетчика на 1), полное удаление товара из корзины (удаление id товара из корзины), счетчик итоговой стоимости, а так же кэширование наполнение корзины (local storage).

```
src > contexts > CartProvider.jsx > ...
1  import { useEffect, useState } from "react";
2  import { CartContext } from "../CartContext";
3
4  export function CartProvider({ children }) {
5    // save to local storage
6    const [cart, setCart] = useState(() => {
7      const savedCart = localStorage.getItem("cart");
8
9      return savedCart ? JSON.parse(savedCart) : [];
10   });
11
12   useEffect(() => {
13     localStorage.setItem("cart", JSON.stringify(cart));
14   }, [cart]);
15
16   function addToCart(item) {
17     setCart((cart) => {
18       const product = cart.find((cartItem) => cartItem.id === item.id);
19
20       if (product) {
21         return cart.map((cartItem) =>
22           cartItem.id === item.id
23             ? { ...cartItem, quantity: cartItem.quantity + 1 }
24             : cartItem
25         );
26       }
27       return [...cart, { ...item, quantity: 1 }];
28     });
29   }
30
31   function removeOneFromCart(id) {
32     setCart((cart) => {
33       cart
34         .map((item) =>
35           item.id === id
36             ? { ...item, quantity: item.quantity - 1 }
37             : item
38         )
39         .filter((item) => item.quantity > 0);
40     });
41   }
42
43   function removeFromCart(id) {
44     setCart((cart) => cart.filter((item) => item.id !== id));
45   }
46
47   const total = cart.reduce((sum, item) => sum + item.price * item.quantity, 0);
48   const cartCount = cart.reduce((sum, item) => sum + item.quantity, 0);
49
50   return (
51     <CartContext.Provider value={{ cart, addToCart, removeOneFromCart, removeFromCart, total, cartCount }}>
52       {children}
53     </CartContext.Provider>
54   );
55 }
56
57
```

Рисунок 8 - Провайдер корзины

5. ThemeProvider.jsx

В ThemeProvider находится логика смены темы на сайте с автоматическим сохранением текущей темы в локальное хранилище браузера.

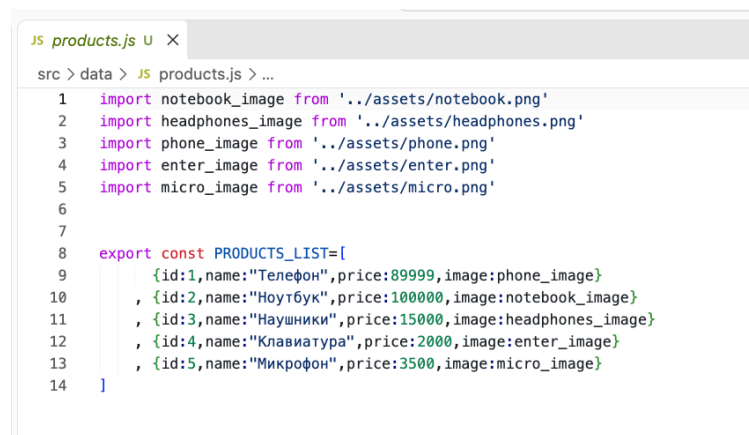


```
1 import { useState } from "react";
2 import { ThemeContext } from "../ThemeContext";
3
4 export function ThemeProvider({ children }) {
5   // save to local storage
6   const [theme, setTheme] = useState(
7     localStorage.getItem("theme") || "light"
8   );
9
10  function toggleTheme() {
11    const newTheme = theme === "light" ? "dark" : "light";
12
13    setTheme(newTheme);
14    localStorage.setItem("theme", newTheme);
15  }
16
17  return (
18    <ThemeContext.Provider value={{ theme, toggleTheme }}>
19      <div className={theme}>
20        {children}
21      </div>
22    </ThemeContext.Provider>
23  );
24 }
25
```

Рисунок 9 - Логика смены темы

6. Product.js

В Product.js находится база данных с товарами откуда берутся данные для вывода в формате карточек на сайт.

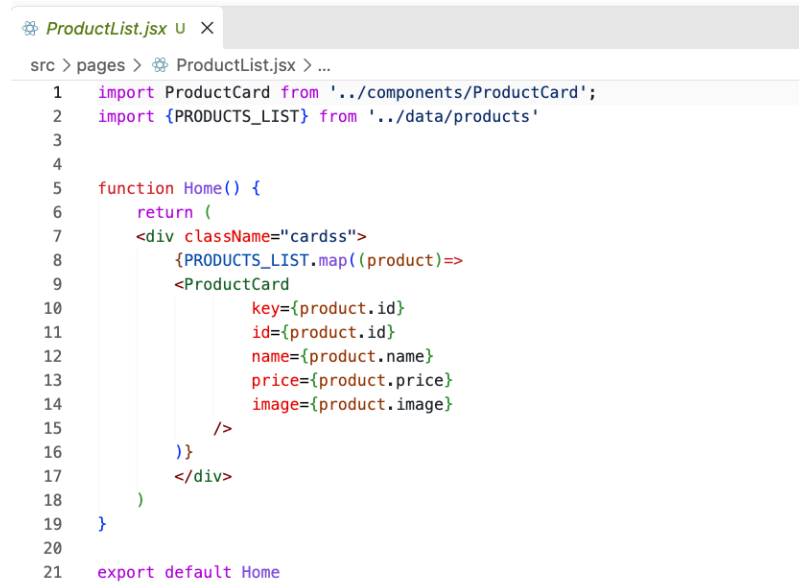


```
1 import notebook_image from '../assets/notebook.png'
2 import headphones_image from '../assets/headphones.png'
3 import phone_image from '../assets/phone.png'
4 import enter_image from '../assets/enter.png'
5 import micro_image from '../assets/micro.png'
6
7
8 export const PRODUCTS_LIST=[
9   {id:1,name:"Телефон",price:89999,image:phone_image}
10  , {id:2,name:"Ноутбук",price:100000,image:notebook_image}
11  , {id:3,name:"Наушники",price:15000,image:headphones_image}
12  , {id:4,name:"Клавиатура",price:2000,image:enter_image}
13  , {id:5,name:"Микрофон",price:3500,image:micro_image}
14 ]
```

Рисунок 10 - База данных с товарами

7. ProductList.jsx

ProductList – главная страница сайта, на которой находятся карточки с товарами. Тут при помощи стрелочной функции происходит перебор данных из БД с товарами и вывод их в формате карточек.



```
ProductList.jsx ×
src > pages > ProductList.jsx > ...
1  import ProductCard from '../components/ProductCard';
2  import {PRODUCTS_LIST} from '../data/products'
3
4
5  function Home() {
6    return (
7      <div className="cardss">
8        {PRODUCTS_LIST.map((product)=>
9          <ProductCard
10             key={product.id}
11             id={product.id}
12             name={product.name}
13             price={product.price}
14             image={product.image}
15           />
16        )}
17      </div>
18    )
19  }
20
21  export default Home
```

Рисунок 11 - Главная страница

8. ProductCart.jsx

ProductCart – страница с корзиной товаров. Тут проверяется есть ли товары в корзине, если нет то выводиться сообщение что корзина пуста, а если есть то выводиться список товаров находящихся в списка товаров в корзине с кнопками изменения количества и удаления товара.

```
ProductCart.jsx u x
src > pages > ProductCart.jsx > ...
1 import { useContext } from "react";
2 import { CartContext } from "../contexts/CartContext";
3
4 function ProductCart() {
5   const { cart, addToCart, removeOneFromCart, removeFromCart, total } = useContext(CartContext);
6
7   return (
8     <div className="cart">
9       <h2>Корзина</h2>
10
11       {cart.length === 0 ? (
12         <p>Корзина пуста</p>
13       ) : (
14         <div>
15           {cart.map((item, index) => (
16             <div className="cart-item" key={index}>
17               <img src={item.image} alt={item.name} />
18               <div className="cart-item-info">
19                 <p>{item.name} - {item.price} руб.</p>
20                 <div className="cart-quantity">
21                   <button className="beautiful-button" onClick={() => addToCart(item)}></button>
22                   <span>{item.quantity}</span>
23                   <button className="beautiful-button" onClick={() => removeOneFromCart(item.id)}></button>
24                 </div>
25               </div>
26             <button className="beautiful-button" onClick={() => removeFromCart(item.id)}>
27               Удалить
28             </button>
29           </div>
30           </div>
31         )}
32       <h3>Итого: {total} руб.</h3>
33     </div>
34   );
35 }
36
37 export default ProductCart;
```

Рисунок 12 - Страница корзины

9. Login.jsx

Login – страница-заглушка с псевдо входом в аккаунт сделанная для редиректа на страницу дашборда.

```
Login.jsx u x
src > pages > Login.jsx > ...
1 import { useNavigate } from "react-router-dom";
2
3 export default function Login() {
4   const navigate = useNavigate();
5
6   function handleLogin() {
7     const isAuth = true;
8
9     if (isAuth) {
10       navigate("/dashboard");
11     } else {
12       alert("Ошибка авторизации");
13     }
14   }
15
16   return (
17     <div style={{ textAlign: "center", marginTop: "120px" }}>
18       <h2>Страница входа</h2>
19       <button className="beautiful-button" onClick={handleLogin}>
20         Войти
21       </button>
22     </div>
23   );
24 }
```

Рисунок 13 - Страница входа

10. Dashboard.jsx

Dashboard – это страница предназначенная для редиректа на нее после ВХОДА.

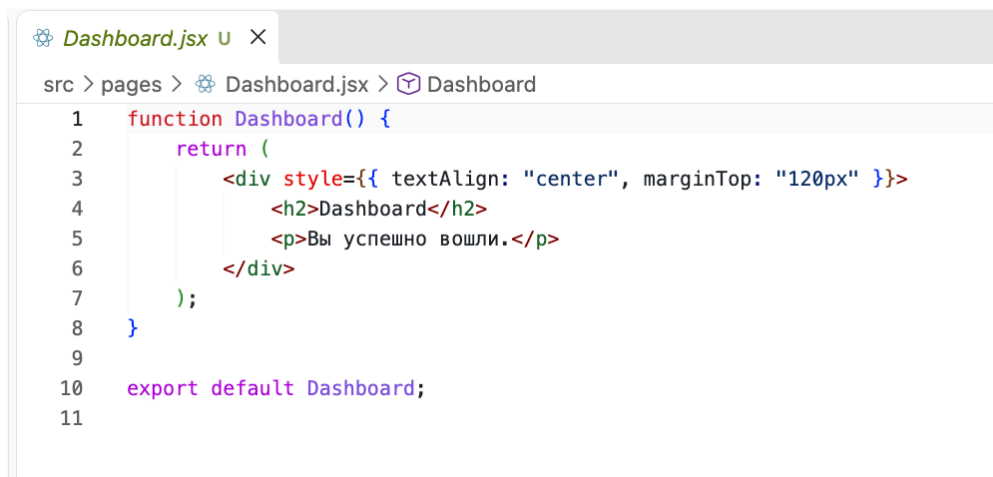


Рисунок 14 - Страница дашборда

11. App.jsx

В App собираются и отображаются описанные ранее страницы, а так же прописаны пути для навигации.

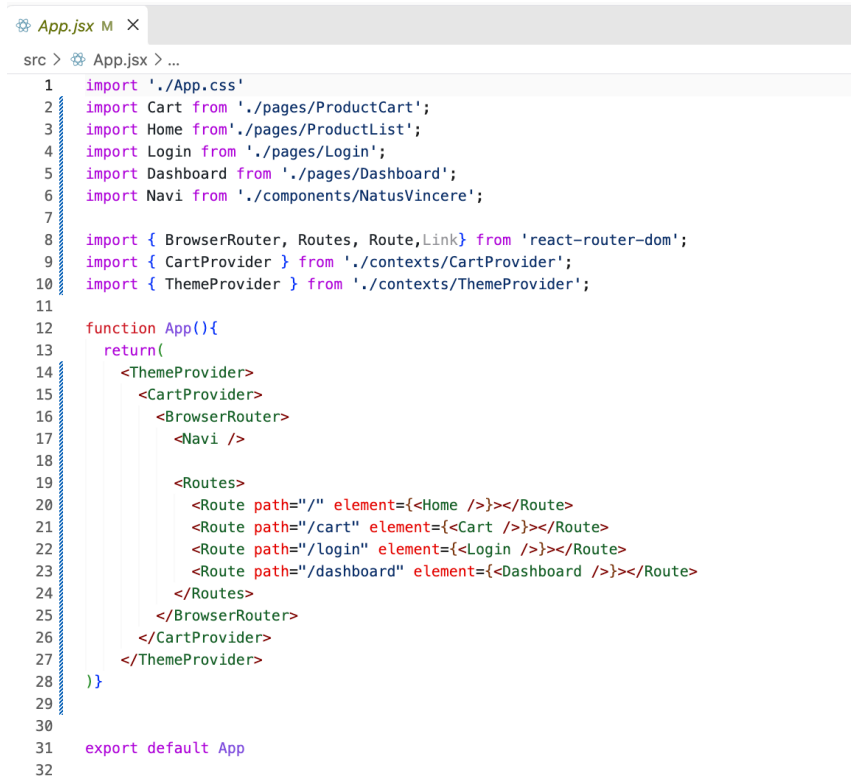


Рисунок 15 - Объединяющий элемент

Вывод: было создано SPA-приложение на React с каталогом товаров, глобальной корзиной и сменой темы.